

基于进迭 K1 开发板的 SSH 远程 LED 智能控制系统

摘要

本项目设计并实现了一款运行于进迭时空 K1 (Muse Pi Pro) RISC-V 开发板的智能语音灯控助手“小宝”。系统以语音控制 LED 灯光为核心，通过软件 PWM 实现无级调光，并集成 BH1750 光感自动调光、声纹安全锁、智能提醒、AI 对话、实时天气与百科搜索、离线模式等功能。硬件采用 USB 麦克风、扬声器、7 寸触摸屏及 BH1750 环境光传感器，软件基于 Python/Tkinter，部署了 SenseVoice 语音识别、MeloTTS 语音合成和 sherpa-onnx 声纹识别等模型，AI 对话通过 DeepSeek API 实现智能回复及联网搜索。灯光控制响应时间 $<1.5\text{ s}$ ，声纹验证准确率达 85% 以上（安静环境），系统功耗约 5 W。项目为 RISC-V 在智能家居、嵌入式 AI 教学等领域的应用提供了实用化参考。

第一部分 作品概述

1.1 功能与特性

1. 语音灯光控制

通过 GPIO 71 软件 PWM，支持“开灯/关灯”“亮一点/暗一点”“开到百分之七十”“最亮/最暗”等自然语言指令，LED 亮度 0- 100%无级调节。灯光状态实时反馈，响应延迟 $<0.5\text{ s}$ 。

2. 声纹安全锁

支持三遍语音注册，存储多个声纹模板，验证时自动取最高匹配分。可设置“仅声纹用户可用”模式，非授权用户无法控制灯光，匹配度低于阈值（默认 0.25）会被拒绝并提示百分比。

3. 智能提醒/闹钟

支持相对时间（“30 秒后提醒我喝水”）和绝对时间（“下午 3 点叫我”“明天早上 10 点提醒我开会”）。屏幕实时倒计时，到点全屏大字“🕒 时间到!”闪烁提醒，同时语音播报和蜂鸣器报警。说“取消提醒”可即时终止，重复提醒每 5 秒一次。

4. AI 智能对话与联网查询

唤醒后可与 AI 自由聊天，接入 DeepSeek API（deepseek-chat 模型），回答简洁友好。支持多轮对话记忆、上下文理解，可直接回答天气、百科、新闻等实时问题，所有信息均由大模型自身知识或推理提供，无需额外调用外部 API。同时支持文本指令控制灯光和提醒。

5. 离线模式

自动检测网络状态（nmcli+ 百度探测），离线时灯光、音量、提醒等本地功能完全正常，联网功能会提示“当前网络不可用”。

7. TTS 语音反馈

MeloTTS 自然人声合成，加入词语级多音字修正（如“重试”→“chong2 shi4”），播放后强制 0.8 s 静默期防止回声，并具备子进程崩溃自动重启机制。

8. 触摸屏 GUI

全屏界面，左上角实时显示日期、星期、节日及在线/离线状态，集成对话日志、音量滑块、WiFi 管理、声纹注册/删除按钮。

9. 环境光自动调光

集成 BH1750 数字光照传感器，实时采集环境光照数值。支持语音 / 屏幕开关自动调光模式，核心控制逻辑：环境亮度越高，LED 输出亮度越低；环境光线越昏暗，灯光亮度自动拉高。系统设置亮度阈值，光线充足时维持最低背光，暗光环境自动拉满照明；手动执行开灯、关灯、固定亮度指令时，自动调光功能临时屏蔽，不干扰手动操作；7 寸触摸屏实时显示当前环境光照值与 LED 输出百分比，直观展示动态调光效果。

1.2 应用领域

1. 智能家居灯控入口：作为桌面语音助手，通过语音控制灯光，替代传统开关，后续可扩展多路灯具和继电器模块控制家电。
2. 嵌入式 AI 教学平台：基于 RISC-V 和 Python 开源生态，展示语音识别、PWM 控制、TTS、声纹识别、API 集成等技术在端侧的完整部署流程，适合高校嵌入式系统、人工智能课程实验。
3. 辅助陪伴助手：为老年人或视障人士提供语音控灯和提醒服务，声纹锁保障使用安全，大字闹钟和语音反馈提升易用性。
4. 物联网语音节点：可扩展温湿度传感器、红外发射模块等，成为全屋语音控制中心，未来可接入 MQTT 等协议实现智能家居联动。

主要技术特点软件 PWM 调光：纯 Python 多线程实现 GPIO 71 的 200 Hz 软件 PWM，占空比 0- 100%可调，无需专用硬件 PWM 外设，代码可移

植性强。

1. 自然语言灯光指令解析：基于关键词匹配和正则表达式，支持“开灯/关灯/开到百分之七十/最亮/最暗/把亮度调高”等十余种变体。
2. 声纹安全锁：基于 sherpa-onnx SpeakerEmbeddingExtractor，注册时采集三遍语音并存储多个模板，验证时遍历所有模板取最高余弦相似度，阈值动态可调。
3. 单录音流复用：声纹注册与验证复用主监听循环的同一段音频，不创建第二个录音实例，避免 K1 USB 音频驱动的 ALSA 冲突。
4. 播后静默期机制：TTS 播放完毕后强制 0.8 s 静默并锁定录音，防止扬声器回声被录入导致误触发。
5. ASR 误识别实时校正：内置纠错字典，将“醒我”自动校正为“提醒我”，持续扩充以提高指令识别成功率。
6. 离线模式自动切换：通过 nmcli 实时检测网络状态，动态切换在线/离线 UI 和功能，离线时本地功能无缝运行。
7. 多源联网信息聚合：

在联网模式下，系统通过 HTTP 请求调用多个免费 API 以获取外部信息。核心 AI 对话能力由进迭云平台的大模型 API（Qwen3- 0.6B）提供，实现零成本、低延迟的智能回复。同时，实时天气数据来源于 wtrr.in 免费接口，百科知识搜索通过 DuckDuckGo 免费接口获取，这些信息类查询结果会交由 AI 进行口语化润色后播报，使离线语音助手在联网时获得丰富且自然的信息能力

1.3 主要性能指标

表 1 系统主要性能指标

指标	数值
语音灯控响应时间	<1.5 s（唤醒+识别+执行+语音反馈）
LED 亮度调节范围	0- 100%，100 级（200 Hz 软件 PWM）
TTS 单句合成延迟	~2.8 s（MeloTTS 整句合成）

ASR 推理 RTF	≤ 0.55 (SenseVoice 量化模型)
唤醒成功率	$> 90\%$ (安静环境)
声纹验证速度	$< 1\text{ s}$ (单次推理)
BH1750 环境光采样频率	1 次 / 0.5s, 实时动态调光

1.4 主要创新点

1. 基于 RISC- V 的端侧语音灯控系统：在 K1 板上完整实现“语音→指令→PWM 灯光控制”闭环，验证了 RISC- V 在智能家居场景的实用性。
2. 软件 PWM 替代硬件 PWM：利用 Python 多线程实现 200 Hz PWM 调光，降低硬件门槛，代码可移植到任何支持 GPIO 的平台。
3. 单录音流声纹复用方案：在不支持多录音流的 K1 USB 音频驱动上，通过复用主监听音频数据实现声纹注册与验证，避免系统崩溃。
4. 多源联网信息聚合：使用 deep seekAPI，使离线语音助手在联网时获得丰富信息能力。
5. ASR 误识别动态校正：根据实际使用场景扩充纠错字典，提升提醒、灯控等关键指令的识别成功率。
6. BH1750 动态光感联动调光：基于环境光照实时数值反向控制 PWM 占空比，光线越强灯越暗，实现无人工干预的自适应护眼照明。

1.5 设计流程

系统设计遵循“硬件选型→驱动适配→模型部署→功能集成→交互优化”的流程：

1. 硬件选型：选择 K1 开发板（RISC- V + 4 核 CPU），搭配 USB 音频外设和面包板 LED。
2. 驱动适配：配置 GPIO、ALSA 音频设备、HDMI 触摸屏，验证外设可用性。
3. 模型部署：逐一部署 SenseVoice ASR、MeloTTS TTS、sherpa- onnx 声纹嵌入模型，解决 ONNX Runtime 兼容性。
4. 功能集成：开发 Tkinter 全屏 GUI，将灯控、提醒、声纹、天气、新闻、搜索等模块整合为多线程+子进程架构。
5. 交互优化：引入播后静默期、ASR 纠错、单流声纹复用、TTS 重启机制等，提升用户体验和系统稳定性。

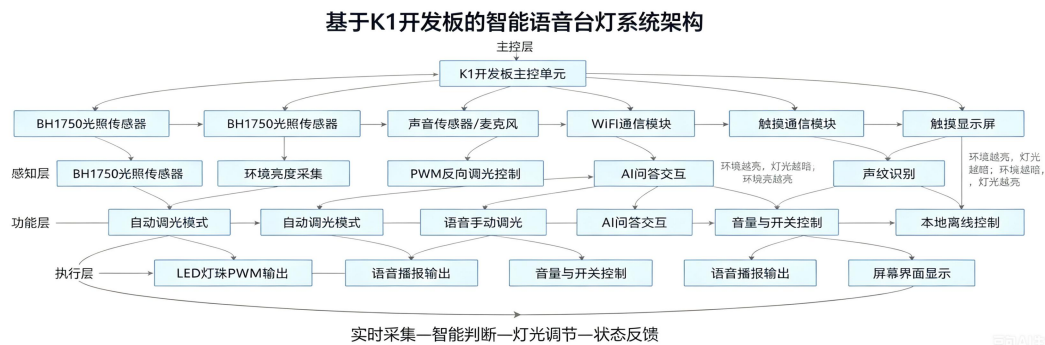


图 1-1 系统设计开发流程图

第二部分 系统组成及功能说明

系统整体架构分为四层：

- 外设层：USB 麦克风、USB 扬声器、HDMI 触摸屏、LED 灯珠（GPIO 71 PWM 调光）、BH1750 I2C 光照传感器（采集环境亮度，反向联动 LED 亮度）
- 核心板层：进迭时空 K1（Muse Pi Pro），玄铁 C910 RISC-V 处理器，1.6 GHz，4 GB RAM，运行 Bianbu OS（基于 Ubuntu 23.04）。
- 软件引擎层：ASR 识别引擎（SenseVoice）、AI 对话客户端（智谱 API）、声纹验证服务（sherpa-onnx 独立进程）、TTS 合成子进程（MeloTTS）、天气/新闻/搜索 API 模块。
- 交互层：Tkinter 全屏 GUI，包含状态栏、对话日志、控制按钮、音量滑块、倒计时显示、声纹管理。
- 各层之间通过管道、HTTP 请求和命令行调用进行通信，主程序作为调度中心协调各模块工作。

整体介绍

系统以进迭时空 K1 开发板为核心，采用四层架构：外设层、核心板层、软件引擎层、交互层。各层通过管道、HTTP 请求和命令行调用通信，主程序负责调度所有模块。系统整体框图如图 2-1 所示。

数据流说明：用户语音由 USB 麦克风采集，经 ASR 引擎识别为文本指令。指令解析器根据关键词和正则表达式分类，分发至灯光控制、提醒设置或 AI 对话模块。灯光控制通过 GPIO 71 的软件 PWM 直接驱动 LED；所有 AI 对话及联网查询（天气、百科、新闻等）均通过 HTTP 请求 DeepSeek API 完成，由大

模型直接回答，无需额外数据源。所有文本回复经 TTS 子进程合成为语音，由 USB 扬声器播放。声纹验证复用主录音数据，在 ASR 线程内完成特征提取与比对，结果影响灯光指令的执行权限。GUI 通过 Tkinter 事件循环刷新状态、日志和倒计时。

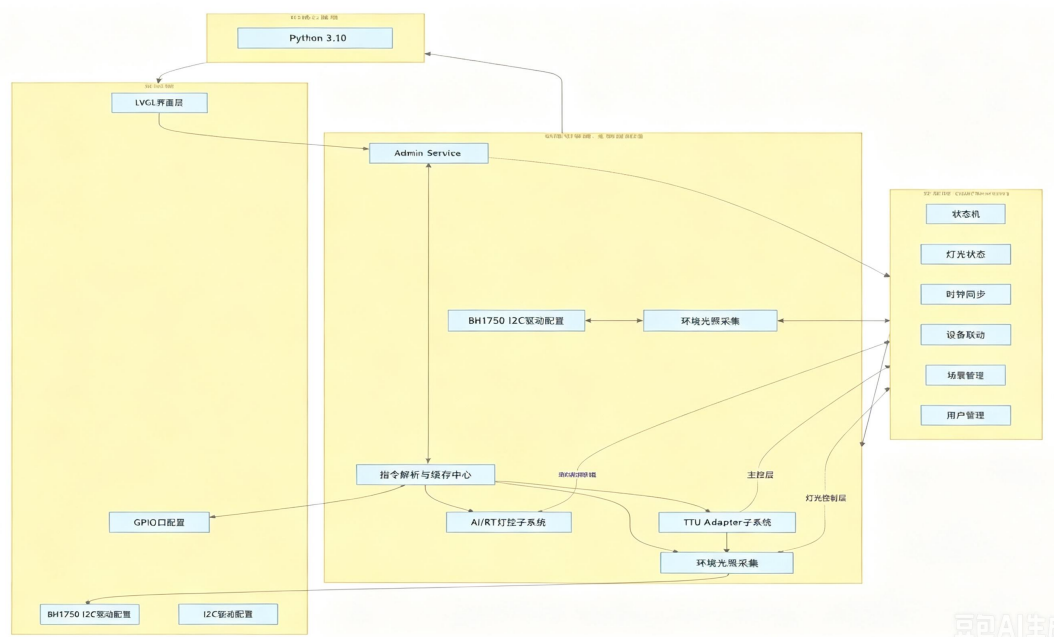


图 2-1 系统整体架构框图

硬件系统介绍

2.2.1 硬件整体介绍；核心硬件为进迭时空 K1（Muse Pi Pro）开发板，外设包括：

- USB 接口麦克风(UACDemoV1.0)和 USB 扬声器(USB PnP Sound Device)，采样率 48 kHz。
- 7 寸 HDMI 触摸屏（1024×600，USB 触控），作为交互界面和显示输出。
- 面包板 LED 灯珠（GPIO 71 驱动，串联 220 Ω 限流电阻），实现软件 PWM 调光。
- BH1750 数字光照传感器（I2C 接口），用于采集环境亮度。

2.2.2 机械设计介绍

本项目不涉及机械结构设计，采用桌面平铺布局：开发板置于底部，面包板

固定于一侧，触摸屏通过 HDMI 和 USB 线缆连接，整体整洁紧凑，便于展示和携带。

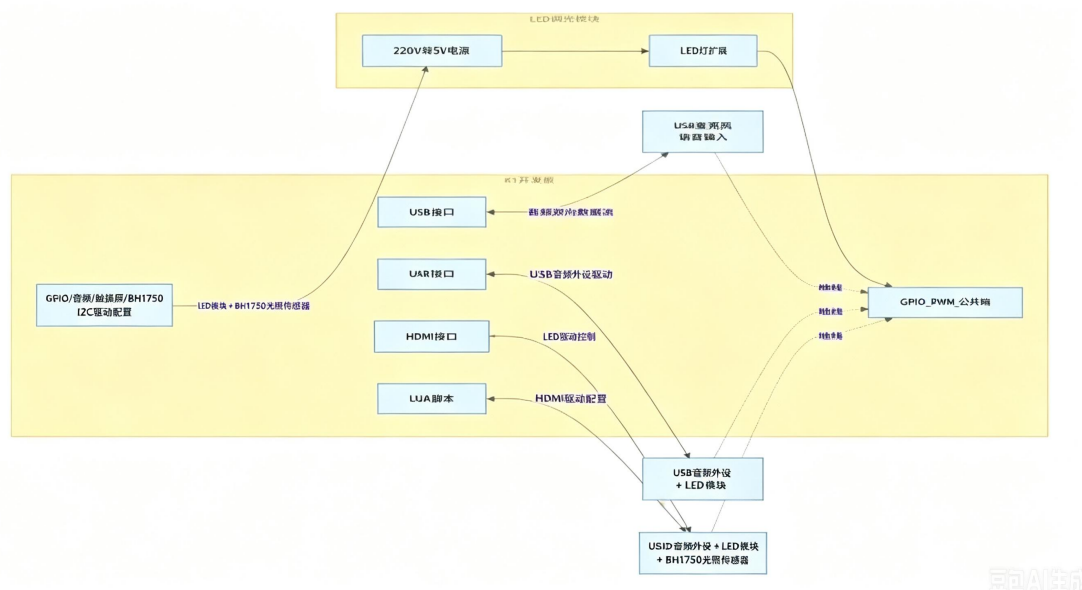


图 2-2 系统硬件连接示意图

2.2.3 电路各模块介绍

LED 调光电路：

K1 GPIO 71（物理引脚 11）→ 220 Ω 电阻 → LED 阳极 → LED 阴极 → GND（物理引脚 6）。

通过软件 PWM 控制 GPIO 高低电平占空比，实现 LED 亮度 0- 100%平滑调节。

BH1750 光感电路：

采用数字光照传感器 BH1750，其 VCC 接开发板 3.3V（物理引脚 1），GND 接 GND（物理引脚 6），SCL 接物理引脚 5，SDA 接物理引脚 3，ADDR 接 GND（物理引脚 6）以设置器件地址为 0x23。通过 I2C 总线读取环境光照度数据，供自动调光算法使用。

1.6 软件系统介绍

2.3.1 软件整体介绍

主程序采用 Python 3.12 + Tkinter 构建，运行于`/root/lianwang/`虚拟环境。整体架构为多线程+子进程模式：

主线程：GUI 事件循环和界面刷新，定时更新时钟和网络状态。

ASR 监听线程：持续录音→VAD 静音检测→ASR 推理→结果回调，同时在此线程内完成声纹验证和注册。

AI 请求线程：发送对话历史到智谱 API，接收回复。

TTS 子进程：常驻 MeloTTS 进程，通过管道接收文本合成语音并播放，播放完毕后发送`PLAYBACK\_DONE`信号。

声纹服务子进程：基于 sherpa-onnx 独立运行于`/root/vp-env/`虚拟环境，通过命令行接口通信，支持 register/list/delete/verify 操作。

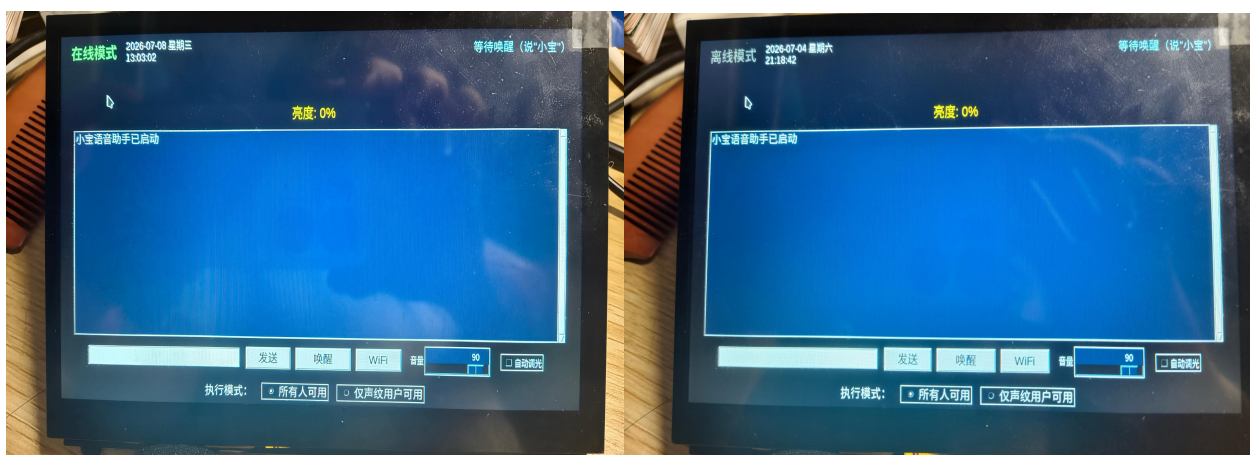


图 2-3 系统主界面截图

2.3.2 软件各模块介绍

(1) 语音识别模块

采用 SenseVoice 量化 ONNX 模型，录音线程基于 VAD（静音检测）截取有效语音段，送 ASR 推理。返回文本经纠错字典校正后，送入指令解析器。唤醒词“小宝”支持同音变体（小吧、小八、小把等），未唤醒时忽略所有非唤醒词语音。

(2) 指令解析与执行模块

`understand(text)`函数基于关键词匹配和正则表达式，将用户意图分为灯光控制（light）、音量调节（volume）、AI 对话（chat）三类。灯光控制支持具体亮度值（如“百分之七十”）和极值（“最亮”“最暗”），以及相对调节（“亮一点”“暗一点”“把亮度调高”）。提醒指令通过`parse\_reminder(text)`和`parse\_absolute\_reminder(text)`提取时间和事件，启动`threading.Timer`定时触发。天气、新闻、搜索通过 API 获取数据，再交由 AI 润色为口语化回复。若开启自

动调光模式，程序循环读取 BH1750 光照数据，实时换算对应 LED 占空比，自动更新 PWM 输出；用户下达手动灯光指令后，自动调光逻辑暂停，直到再次语音开启自动调光。

（3）AI 对话模块

`call\_ai\_with\_history(messages)`向 deepseek API（或其他 api）发起 POST 请求，返回内容经`\_speak\_text`送 TTS 播报。系统提示词为“你是小宝，一个可爱的智能助手。回答简洁友好。”，保证回复风格统一。失败时自动重试并返回友好提示。

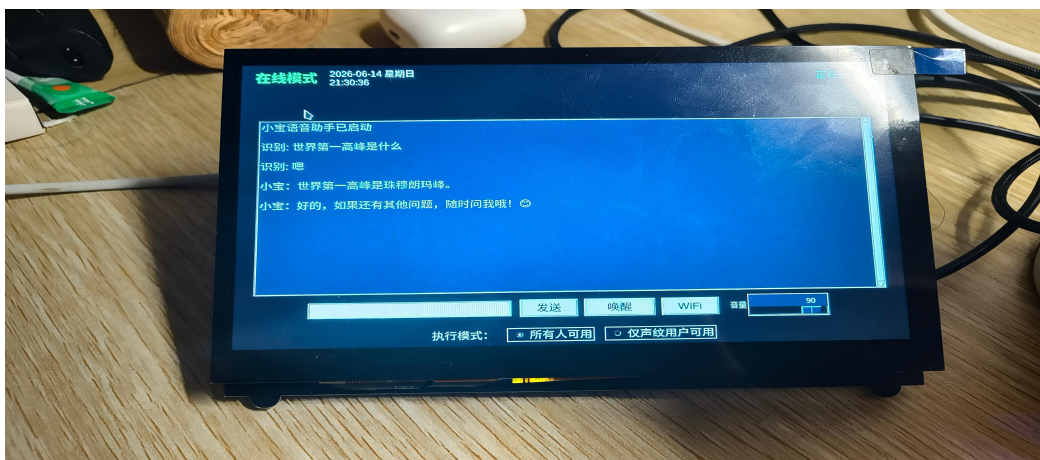


图 2-4 ai 对话界面截图

（4）TTS 模块

MeloTTS 子进程常驻运行，通过标准输入接收文本，合成后调用 aplay 播放。播放完毕后发送`PLAYBACK\_DONE`信号，主程序释放录音锁并触发 0.8 s 静默期，防止回声干扰。同时集成词语级多音字修正字典（如“重试”改为“chong2 shi4”）。若子进程崩溃，可自动重启恢复。

（5）声纹安全模块

`voice\_service.py`独立运行于`/root/vp-env/`虚拟环境，基于 sherpa-onnx SpeakerEmbeddingExtractor。注册时采集三遍语音，两两比对确保一致性后存入数据库。验证时遍历所有模板取最高余弦相似度，与阈值比较决定是否通过。主程序通过`call\_vp()`函数调用其接口，注册和验证均复用主监听音频数据，不创建额外录音流。

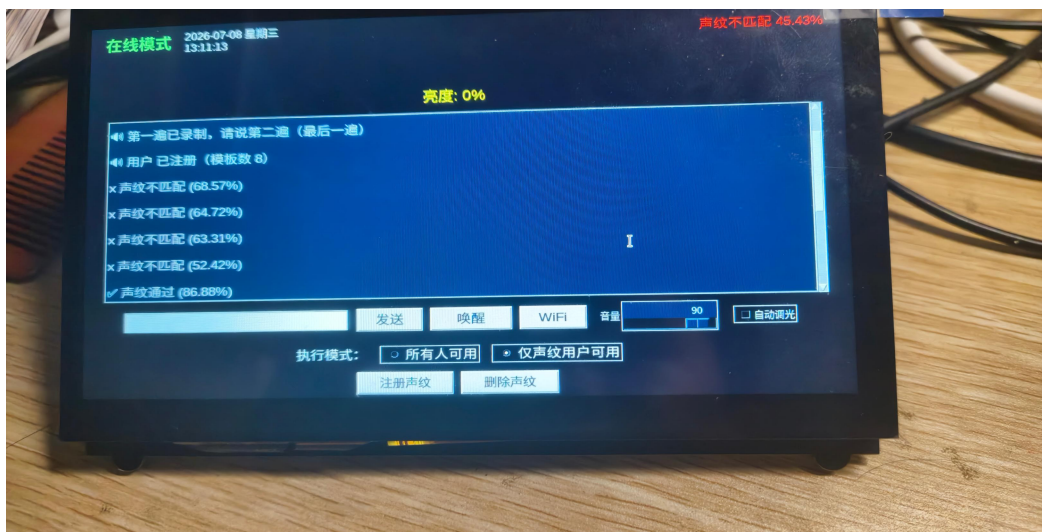


图 2-5 声纹识别界面截图

(6) 联网搜索模块

天气数据或百科知识通过 deep seekAPI (或 wttr.in 免费 API) 获取，这些数据经 AI 润色后以语音和文字形式输出，搜索优先级低于时间查询和天气查询，避免误拦截。

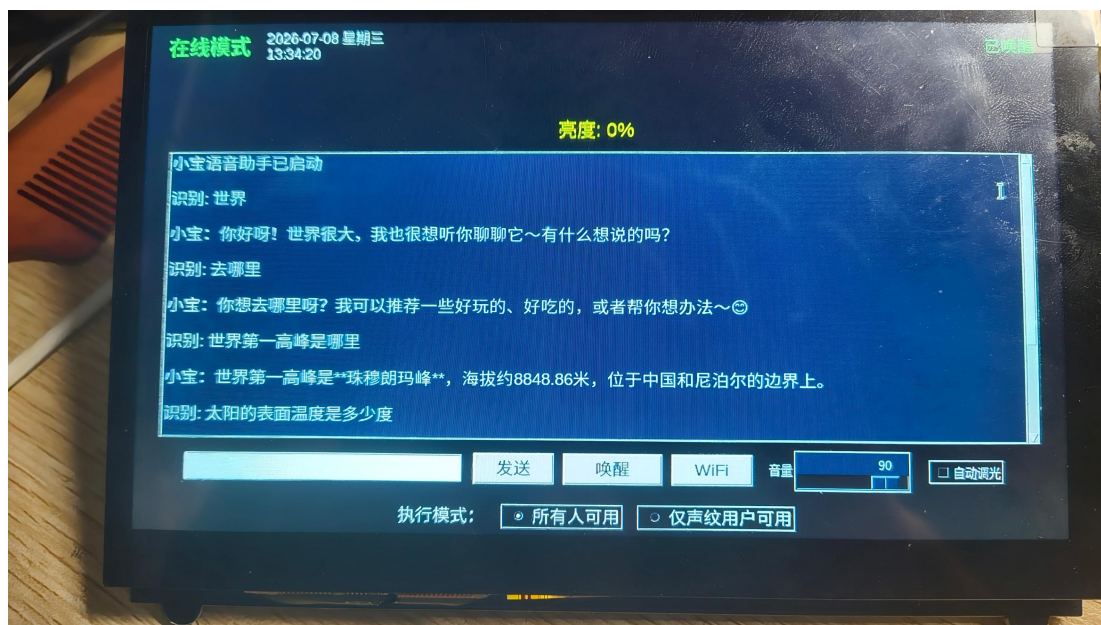


图 2-6 联网搜索界面截图

第三部分 完成情况及性能参数

3.1 整体介绍

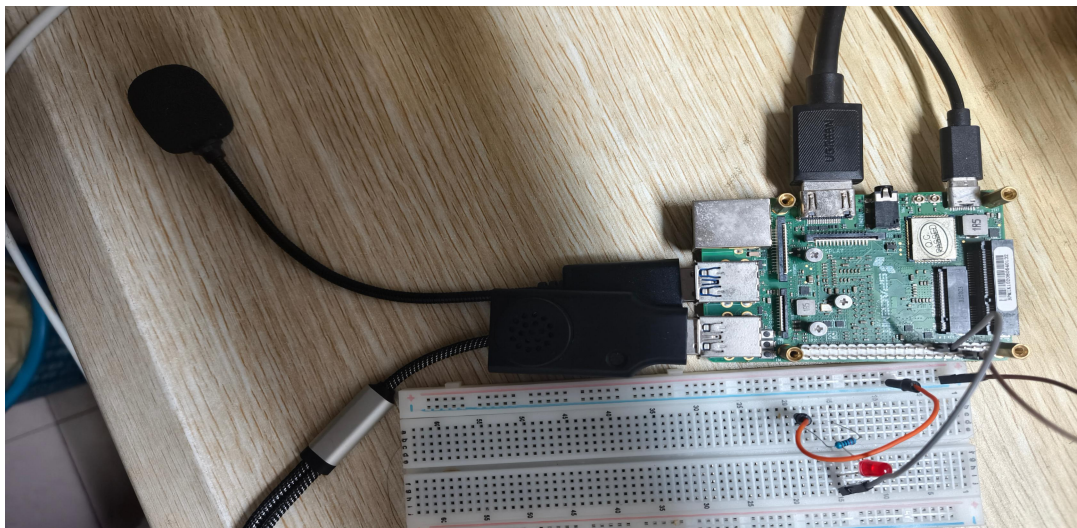


图 3-1 系统实物整体图 1

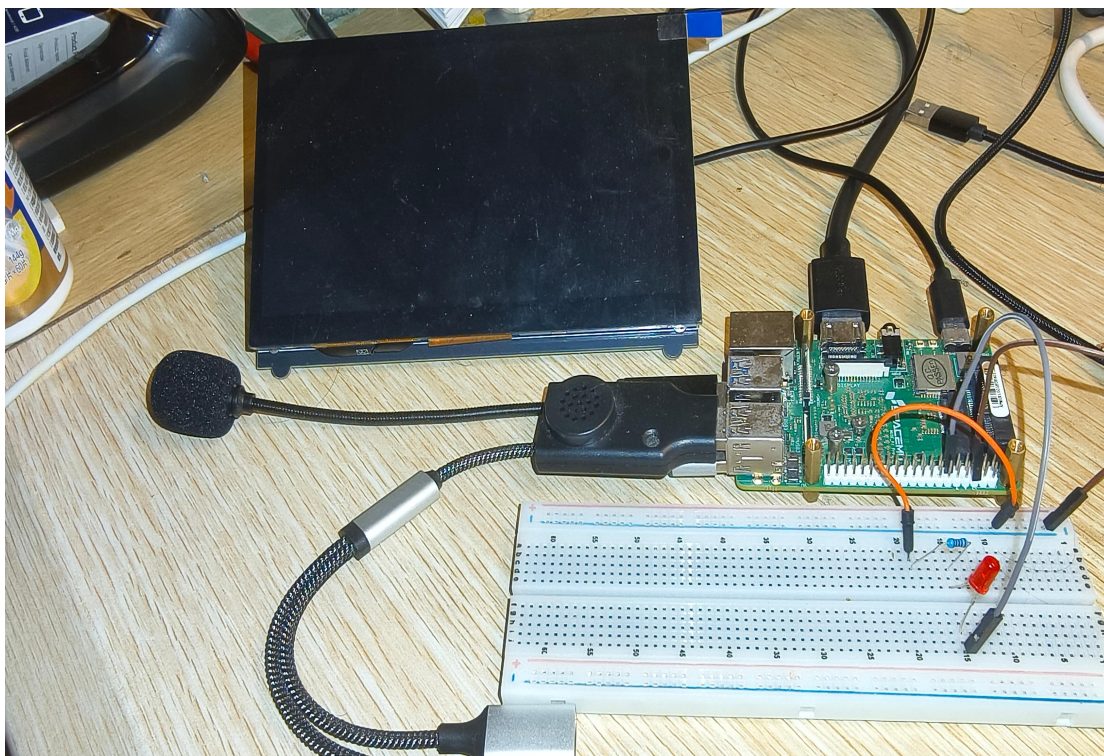


图 3-1 系统实物整体图 2

3.1.1 工程成果

3.1.2 机械成果；

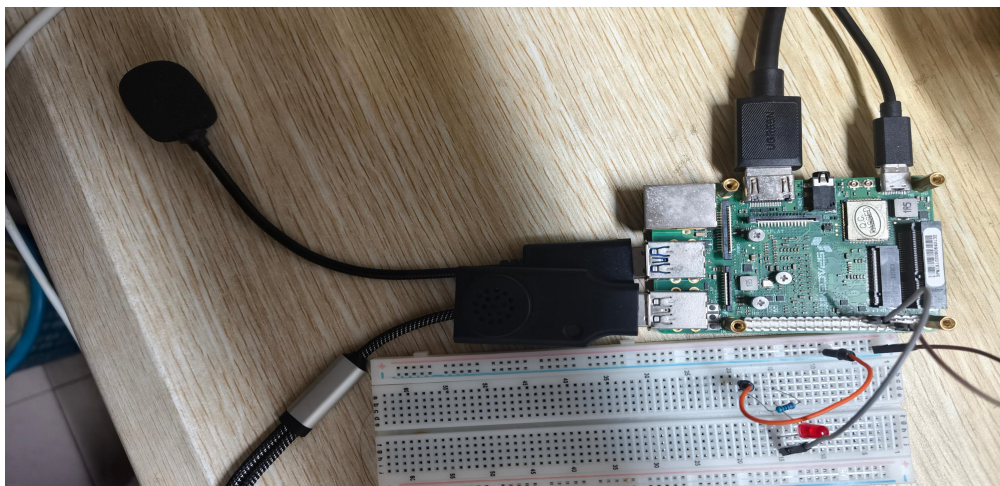


图 3-2 系统硬件实物图

3.2.1 电路成果；

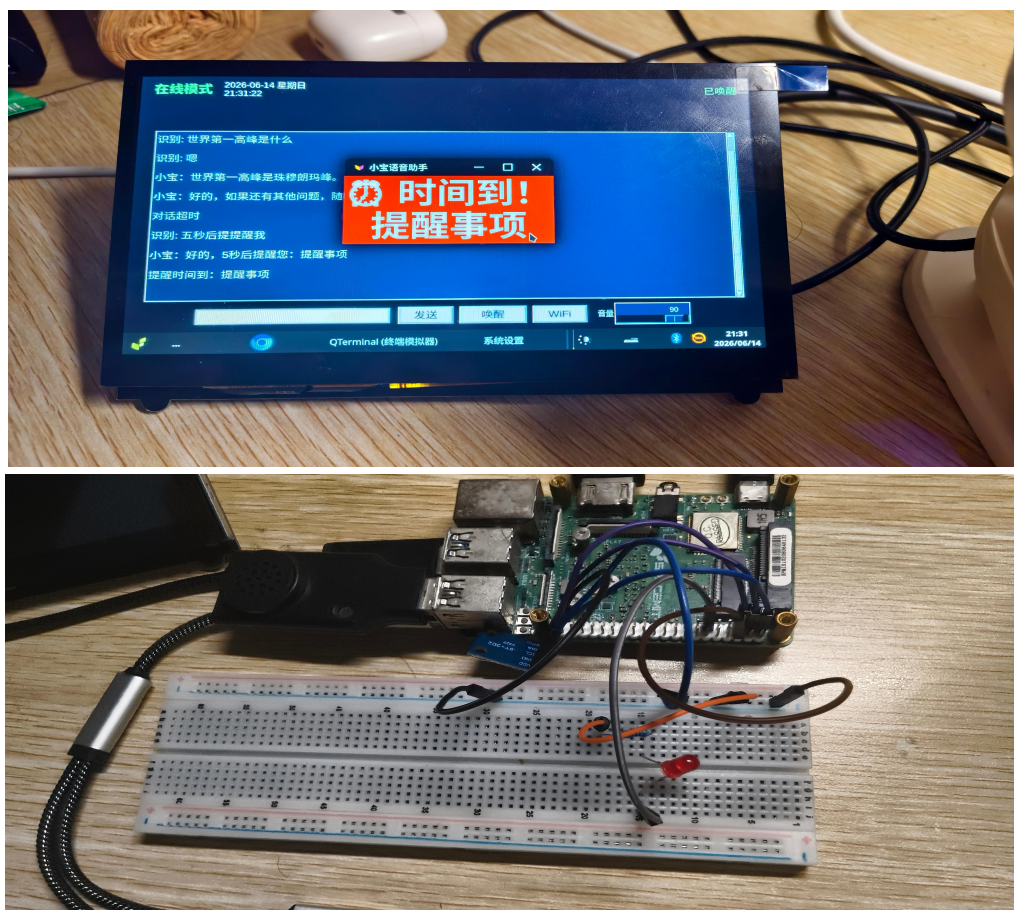


图 3-3 LED 调光电路实物图

3.2.2 软件成果；

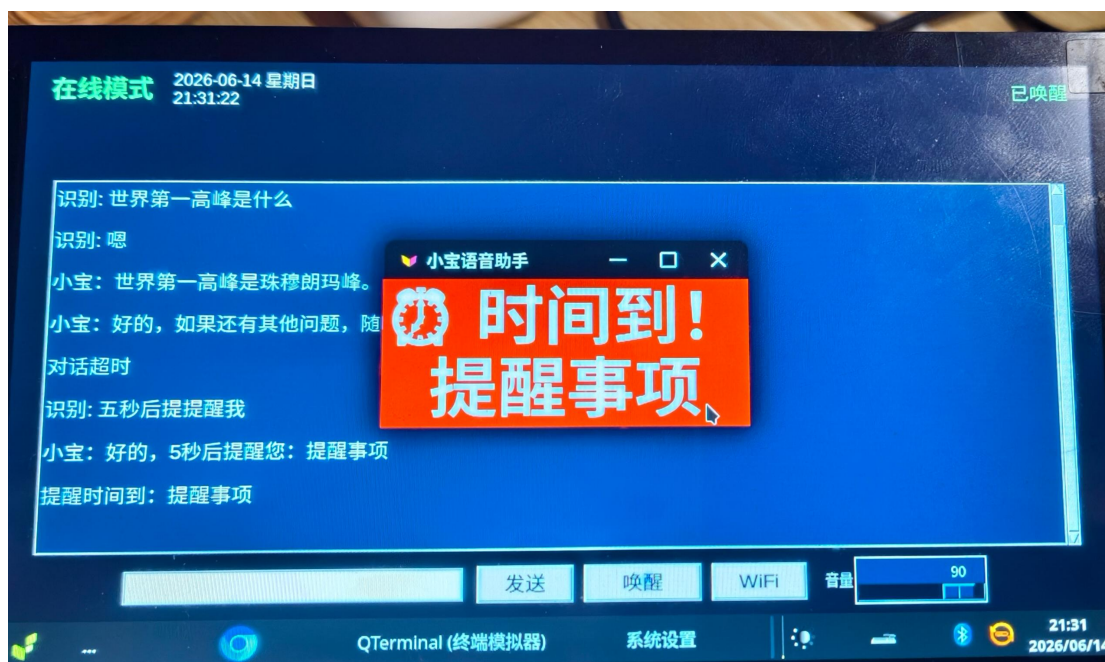


图 3-4 定时提醒功能界面

特性成果

1. 灯光控制测试

说“开灯”，LED 在 $<0.5\text{ s}$ 内从熄灭变为 100%亮度，语音反馈“好的，灯光已调节”约 2.8 s 后播放。

说“开到百分之七十”，LED 精确变为 70%亮度。

说“最亮”，LED 变为 100%亮度。

说“关灯”，LED 熄灭。

2.自动调光功能测试：开启自动调光后，遮挡 BH1750 传感器、环境变暗，LED 灯光自动变亮；用强光照传感器、环境变亮，LED 灯光逐步变暗，屏幕同步实时刷新环境亮度与 LED 输出百分比，调光过渡平滑无频闪。

2. 声纹安全测试*

注册本人声纹（三遍）后，切换到“仅声纹用户可用”模式，本人说“开灯”正常执行，屏幕显示“✓ 声纹通过 (XX%)”。

其他人说同样指令，屏幕提示“✗ 声纹不匹配 (XX%)”并忽略，灯光不动作。

3. 提醒功能测试

说“30 秒后提醒我喝水”，屏幕显示倒计时，30 秒后播报“您设置的时间到了”，全屏显示“🕒 时间到！”，语音播报响三声。

说“下午 3 点叫我”，在设定时间触发同样提醒。

说“取消提醒”，倒计时消失，重复播报和闹钟停止。

4. 天气、新闻、搜索测试

说“今天北京天气怎么样”，通过 API(deepseek)获取北京实时天气并口语化播报。

说“今天有什么新闻”，DeepSeek 模型根据知识库提供简要新闻摘要。

说“搜索珠穆朗玛峰有多高”，DeepSeek 模型返回百科类知识回答。

5. 离线模式测试

断开 WiFi，左上角变为灰色“离线模式”。

说“开灯”，灯光正常响应。

说“今天天气怎么样”，语音提示“当前网络不可用”。

-说“30 秒后提醒我”，倒计时正常，到点提醒正常。

第四部分 总结

1. 多路灯光与家电控制：增加 GPIO 引脚和继电器模块，实现多路灯具独立语音控制，扩展至空调、电视等红外遥控家电。
2. NPU 加速：若能适配 K1 板载 NPU 执行提供者，TTS 动态合成延迟有望从 2.8 s 降至 1 s 以内，AI 对话也可本地运行。
3. 流式 ASR：替换为 whisper.cpp 或 Paraformer 流式推理，实现边说边出字，进一步提升交互实时性。
4. 本地大语言模型：若 K1 资源允许，部署轻量级端侧 LLM（如 Qwen2.5-0.5B），实现完全离线 AI 对话。
5. 更多物联网外设：扩展温湿度传感器、红外发射模块等，通过语音查询室内环境并联动控制家电。

4.1 心得体会

本项目从零开始，在 RISC-V 开发板上完成了智能语音灯控助手的完整开发，过程充满挑战与收获。

模型部署是最大难点。K1 的 RISC-V 架构在 ONNX Runtime 兼容性方面资料匮乏，团队花了大量时间测试不同版本的 onnxruntime 和模型量化参数，最终才让 SenseVoice 和 MeloTTS 稳定运行。声纹模型 sherpa-onnx 的选择和集成也经历了多次尝试，最终采用了独立进程+命令行接口的解耦方案，避开了 Python 依赖冲突。这让我们深刻认识到：嵌入式 AI 落地的关键不在于模型精度，而在于工程适配与系统稳定性。

软件 PWM 的实践价值。我们使用 Python 多线程模拟 200 Hz PWM 调光。虽然增加了 CPU 开销，但完全满足调光需求，且代码可移植到任何支持 GPIO 的平台，降低了硬件门槛。

单录音流方案的诞生。声纹功能最初尝试独立录音引擎，导致 ALSA 驱动频繁崩

溃（Stream not open）。我们最终采用了“复用主监听音频数据”的架构——注册和验证都在主录音流中完成，不创建第二个录音实例。这个经验在资源受限的嵌入式开发中非常宝贵。

用户体验需要“细颗粒打磨”。最初版本 TTS 播放完毕后立即开放录音，导致扬声器回声被录入，造成误识别。我们通过增加 0.8 s 强制静默期和录音锁定机制解决了这一问题。ASR 误识别校正字典也在使用中持续扩充，显著降低了指令失败率。天气和新闻的口语化润色、离线模式的自动切换，都让产品从一个“能用的 Demo”变成了“好用的助手”。

RISC-V 生态虽起步晚，但已具备实用性。本次项目证明：在 4 核 1.6 GHz 可行的，为未来国产自主芯片在智能家居领域的应用提供了有力验证。

最后，感谢开源社区提供的 SenseVoice、MeloTTS、sherpa-onnx、wttr.in 等优秀项目和 API，以及进迭时空官方的技术支持。

第五部分 参考文献

- [1] 进迭时空. Muse Pi Pro 开发板用户手册[EB/OL]. 2024.
- [2] 阿里巴巴达摩院. SenseVoice 语音识别模型[EB/OL]. GitHub, 2024.
- [3] MeloTTS. MeloTTS: 开源多语言文本转语音系统[EB/OL]. GitHub, 2024.
- [4] DeepSeek 团队 . DeepSeek API 文档 [EB/OL]. 2024.
<https://www.deepseek.com/docs>.
- [5] k2-fsa 团队 . sherpa-onnx: 语音处理工具包 [EB/OL]. GitHub, 2024.
<https://github.com/k2-fsa/sherpa-onnx>.
- [6] Microsoft. ONNX Runtime 官方文档[EB/OL]. 2024. <https://onnxruntime.ai/docs/>.
- [7] 进迭时空 . spacemit-demo: K1 AI 示例代码 [EB/OL]. GitHub, 2024.
<https://github.com/spacemit/spacemit-demo>.
- [8] 百度. jieba 中文分词库[EB/OL]. GitHub, 2024. <https://github.com/fxsjy/jieba>.
- [9] Python Software Foundation. Python 3.12 官方文档 [EB/OL]. 2024.
<https://docs.python.org/3.12/>.
- [10] ROHM Semiconductor. BH1750FVI 数字环境光传感器数据手册[EB/OL]. 2011.

- [11] gpiozero Contributors. gpiozero: Python GPIO 库文档[EB/OL]. GitHub, 2024.
<https://github.com/gpiozero/gpiozero>.
- [12] SpacemiT. Bianbu Robot 开发文档 [EB/OL]. (2026-07-05).
<https://www.spacemit.com/community/document/info?lang=zh&nodepath=software/S DK/ros/k1>.